NEOLABS

SECURITY LABS | SOC LEVEL 1

Week 1 Launch Pack — Operation Night Watch

Weekly Learning Pack | Learn + Connect + Operate

Programme: NeoLabs x Renaissance Innovation Labs Cybersecurity Internship
Week 01
Version: 1.0
Classification: Student Training Material
Authorised synthetic training use only.

Your objective

Build a defensible picture of **normal** VCC activity in your assigned pod. This baseline becomes the comparison point for later security incidents.

Start the SOC workstation

Windows

Pull the latest toolkit and either double-click `START-NEOLABS-SOC.cmd` or run from PowerShell:

```
.\START-NEOLABS-SOC.cmd
```

Command Prompt users run:

```
START-NEOLABS-SOC.cmd
```

Linux / Ubuntu

From the toolkit root:

```
bash start-neolabs-soc.sh
```

The platform launcher is responsible for first-run prerequisites and Wazuh preparation. Do not manually run individual files under `internal/` or `wazuh-stack/scripts/` as a substitute setup sequence.

Enter the assigned pod number + private NeoLabs Access Code only if prompted, then wait for:

```
SOC WORKSTATION READY
```

READY means the toolkit verified that a real synthetic event for the **server-assigned pod** reached the local Wazuh data path and is searchable in `wazuh-alerts-*`.

The launcher also reports newest-event freshness, checks local alert retention/disk state, provisions Night Watch/Telemetry Health saved objects when supported and provides dashboard access.

Wazuh login

```
Dashboard: https://127.0.0.1:8443
Username: admin
```

On Windows the password is copied to the clipboard without being printed. On Linux the launcher reports the protected local credential when appropriate; do not post screenshots containing it.

On a supported headless Linux server, the dashboard is published on TCP 8443 and the launcher reports the server URL. Cloud/host firewall rules must restrict access to the intern's approved source IP.

NeoLabs CLI from the repository root

Do not navigate into `tools/`.

Windows PowerShell:

```
.\neolabs.cmd status
.\neolabs.cmd doctor
.\neolabs.cmd login
.\neolabs.cmd connect
```

Linux / Ubuntu:

```
bash neolabs status
bash neolabs doctor
bash neolabs login
bash neolabs connect
```

If startup/telemetry is not healthy

Windows PowerShell:

```
.\neolabs.cmd doctor
```

Linux:

```
bash neolabs doctor
```

Doctor checks NeoLabs authentication → LIVE/REPLAY surface → raw VCC event file → Wazuh rule engine → Filebeat → indexer → dashboard/manager API, plus telemetry freshness and local index/disk state.

Do not solve setup problems by adding `sudo` to random internal Wazuh commands. The platform launcher owns the small number of OS-level privileged operations needed during first setup.

LIVE / REPLAY behaviour

The server decides whether the current authorised SOC surface is LIVE or REPLAY. Live telemetry uses the protected pod-scoped channel; replay loads only validated archived telemetry for the assigned pod/scenario into the same local Wazuh workflow. Original `event_time` is preserved and replay metadata is separate.

What to study before the task

Use the material in `publications/` in this order:

1. **Log Literacy for Cybersecurity Analysts**.
2. **SecOps Foundations / Field Guide**.
3. **SOC L1 Analyst Handbook**.
4. **Wazuh Deployment and Investigation Guide**.
5. **SOC L1 Complete Toolkit** as long-form reference.

Where to work in Wazuh

Use **NeoLabs — Operation Night Watch** when available. Otherwise use Threat Hunting/Discover over `wazuh-alerts-*` and filter the server-assigned pod.

Important fields include:

- `data.pod_id`
- `data.event_type`
- `data.dstuser`
- `data.source_ip`
- `data.outcome`
- `data.correlation_id`
- `rule.id`, `rule.level`, `rule.description`
- original `data.event_time`

Use **NeoLabs — Telemetry Health** for rule `100150` and collector/parser/visibility problems. A zero-result query is not proof of absence until telemetry health/freshness is checked.

Week 1 task

1. Confirm Wazuh shows only the correct assigned pod/scenario context.
2. Record newest VCC event freshness and any telemetry-health warning.
3. Identify normal successful authentication and an ordinary failed authentication if present.
4. Identify normal application/API activity and useful identity/source/outcome/session/correlation fields.
5. Identify storage or other approved telemetry visible for your pod.
6. Create/save at least **three reusable baseline searches/filters**.
7. Build a short timeline using at least two event types and original event time.
8. Record one visibility gap and what source/field would close it.
9. Do not label activity malicious merely because it is unusual; Week 1 is the reference baseline.

Deliverables

- `baseline-log-report.md`
- `timeline.md`
- `evidence-log.md`
- `query-journal.md`
- redacted screenshots where useful

Official graded work goes to `RIL_NeoLabs-Intern-Assignments`, not this toolkit repository.

Evidence standard

- Use original event time for incident sequence and record replay/ingestion/index time separately when relevant.
- Preserve request/event/correlation IDs where available.
- Separate observed facts from interpretation.
- Redact Access Codes, Wazuh passwords, tokens, signed URLs, certificates/private keys and personal data.
- Never include another pod's data.

Stop conditions

Stop and contact a mentor if another pod's events, real personal/production information, credentials/private keys, unexpected infrastructure access or service instability appears.

Before submission

- [ ] `SOC WORKSTATION READY` was reached.
- [ ] Current status shows the correct pod, track and Week 1 scenario.
- [ ] Latest-event freshness/telemetry health was checked.
- [ ] Three baseline searches/filters are recorded.

- [ ] Timeline uses at least two event types and original event time.
- [ ] Evidence is reproducible and redacted.
- [ ] Conclusions are supported by evidence.